

# Gatilhos e Execução

## Gatilhos de Execução (on:)

O fluxo é ativado automaticamente em três situações:

- **push:** Sempre que novas alterações são enviadas para a branch `master`
- **pull\_request:** Sempre que um PR é aberto ou atualizado tendo a branch `master` como destino
- **Filtros de Caminho (paths):** Em ambos os casos acima, o fluxo só roda se as alterações acontecerem dentro da pasta `app/` ou no próprio arquivo de configuração do workflow
- **workflow\_dispatch:** Permite que qualquer desenvolvedor execute o teste manualmente pela interface do GitHub

## Execução dos Testes Unitários

Depois de entrar na pasta do projeto, execute os testes unitários com o flag `--no-daemon` para evitar processos em segundo plano e garantir uma execução mais estável.

# Tarefas (jobs:)

O arquivo define apenas uma tarefa principal chamada "**testes-android**".

- **Ambiente:** Executa em uma máquina virtual Linux atualizada (ubuntu-latest)
- **Nome:** Exibido no GitHub como "Testes Unitários Kotlin"

# Passo 1 – Checkout do Código

Baixa o código-fonte do repositório para dentro da máquina virtual do GitHub Actions.

Este é o primeiro passo essencial que prepara o ambiente com o código que será testado.

```
name: Android CI

on:
  push:
    branches:
      - master
    paths:
      - 'app/**'
      - '.github/workflows/android-ci.yml'
  pull_request:
    branches:
      - master
    paths:
      - 'app/**'
      - '.github/workflows/android-ci.yml'
  workflow_dispatch:

jobs:
  testes-android:
    name: Testes Unitários Kotlin
    runs-on: ubuntu-latest

    steps:
      - name: Checkout do código
        uses: actions/checkout@v4
```

# Passo 2 – Configuração do Java

Instala o Java Development Kit (JDK) 21 usando a distribuição Temurin. Ativa o cache para o Gradle, o que acelera as próximas execuções evitando baixar dependências repetidas.

Este passo garante que o ambiente tenha a versão correta do Java e otimiza o tempo de build.

```
- name: Configurar Java 21
  uses: actions/setup-java@v4
  with:
    distribution: 'temurin'
    java-version: '21'
    cache: 'gradle'
```

```
001-0000
- name: Criar Mock do google-services.json para o compilador
  run: |
    echo '{"project_info":{"project_number":"123456789","project_id":"smart-drain-
mock"},"client":[{"client_info":{"mobilesdk_app_id":"1:1234:android:1234","android_client_info":
{"package_name":"br.edu.fatecpg"}},"api_key":
[{"current_key":"mock_key_for_ci_purposes_only"}],"services":{"appinvite_service":{"status":1}}}]}' >
app/app/google-services.json
```

## Passo 3 – Mock do google-services.json

Cria um arquivo falso (mock) de configuração do Firebase/Google Services. O compilador do Android exige esse arquivo para buildar o app.

Como chaves reais de produção não devem ser expostas no GitHub, injeta-se uma estrutura de texto válida apenas para o compilador durante os testes.

```
- name: Dar permissão de execução ao Gradle
  run: chmod +x app/gradlew

- name: Executar Testes Unitários
  run: |
    cd app
    ./gradlew testDebugUnitTest --no-daemon
```

## Passo 4 – Permissão do Gradle

Altera as permissões do script de execução do Gradle (`gradlew`) para garantir que a máquina virtual tenha autorização para executá-lo.

Este passo é essencial para que o Gradle possa ser executado no ambiente de CI/CD.